

Patterns

Content notes, classwork and homework with worked C++ solutions.

Lesson 1: Mixed Loops, Series & Number Patterns

Quick recap: Mixed loop problems • Series and summations • Simple number patterns

Theory & Nuances

Mixed loop problems

Real problems rarely use just one loop in isolation. A while loop is the natural fit when you don't know the iteration count up front (e.g. peeling digits off a number with $n \% 10$ and $n / 10$ until n becomes 0), while a for loop fits a known count. Many problems combine a loop with an if inside it, or run two loops one after another, each doing a separate job.

Series and summations

Every series problem has the same skeleton: find the formula for the i -th term, keep an accumulator outside the loop, and add $\text{term}(i)$ to it on every iteration. Alternating series ($1 - 2 + 3 - 4 \dots$) just flip the sign based on $i \% 2$. For fractional series like $1 + 1/2 + 1/3$, the accumulator and the division must be done in double.

Simple number patterns

A pattern is a grid: the outer loop walks the rows, the inner loop walks the columns of the current row, and a newline is printed after the inner loop ends. What you print inside decides the pattern — printing j gives $1\ 2\ 3 \dots$, printing i repeats the row number, and printing a counter that never resets gives Floyd's triangle.

Key Points / Common Mistakes

- $1 / i$ with an `int i` is integer division — it becomes 0 for every $i > 1$. Write $1.0 / i$.
- Forgetting the `cout << endl` after the inner loop prints the whole pattern on one line.
- Use a while loop when the stopping point depends on the data (like digits), a for loop when you know the count.

Classwork Problems

1. Sum of digits and reverse of a number

```
#include <iostream>
using namespace std;

int main() {
    int n = 4527;
    int temp = n, sum = 0, reversed = 0;

    while (temp > 0) {
        int digit = temp % 10;
        sum += digit;
        reversed = reversed * 10 + digit;
        temp /= 10;
    }

    cout << "Number: " << n << endl;
    cout << "Sum of digits: " << sum << endl;
    cout << "Reversed: " << reversed << endl;
    return 0;
}
```

Output

```
Number: 4527
Sum of digits: 18
Reversed: 7254
```

2. Sum of squares and alternating series

```
#include <iostream>
using namespace std;

int main() {
    int n = 6;

    // 1^2 + 2^2 + ... + n^2
    int squares = 0;
    for (int i = 1; i <= n; i++) {
        squares += i * i;
    }

    // 1 - 2 + 3 - 4 + ... (+/-) n
    int alternating = 0;
    for (int i = 1; i <= n; i++) {
        if (i % 2 == 1) alternating += i;
        else alternating -= i;
    }

    cout << "Sum of squares up to " << n << ": " << squares << endl;
    cout << "1 - 2 + 3 - ... up to " << n << ": " << alternating << endl;
    return 0;
}
```

Output

```
Sum of squares up to 6: 91
1 - 2 + 3 - ... up to 6: -3
```

3. Number triangle

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cout << j << " ";
        }
        cout << endl;
    }
    return 0;
}
```

Output

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Homework Problems

1. Harmonic series $1 + 1/2 + \dots + 1/N$

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    double sum = 0;

    for (int i = 1; i <= n; i++) {
        sum += 1.0 / i;
    }

    cout << "Sum of harmonic series up to 1/" << n << " = " << sum << endl;
    return 0;
}
```

Output

```
Sum of harmonic series up to 1/5 = 2.28333
```

2. Floyd's triangle

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    int num = 1;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cout << num << " ";
            num++;
        }
        cout << endl;
    }
    return 0;
}
```

Output

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Lesson 2: Basic Star Patterns

Quick recap: Single row / column • Solid square • Right-angled triangle • Inverted triangle

Theory & Nuances

Single row / column

A single row is one loop printing stars with no newline in between; a single column is one loop printing a star followed by a newline every time. This is the smallest building block — every 2D pattern is just rows stacked on top of each other.

Solid square

An $n \times n$ square needs two nested loops: the outer loop runs once per row, the inner loop prints n stars in that row. The inner loop's bound does not depend on i , which is exactly why every row has the same length. A rectangle is the same code with different row and column counts.

Right-angled triangle

Make the inner loop's bound depend on the row number: row i prints i stars ($j \leq i$). Before coding any pattern, write down a small table of row number vs. how many stars it has — the formula for the inner loop falls straight out of it.

Inverted triangle

Flip the table: row 1 has n stars, row 2 has $n-1$, ... row i has $n - i + 1$. Either keep i counting up and print $n - i + 1$ stars, or let the outer loop count down from n to 1 and print i stars — both give the same output.

Key Points / Common Mistakes

- The outer loop always controls rows; the inner loop always controls what goes inside one row.
- Print "*" (star + space) instead of "*" if you want a square to look square in the terminal.
- Make a row/stars table for $n = 4$ before coding — it catches off-by-one errors immediately.

Classwork Problems

1. Single row and single column of stars

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    cout << "Row:" << endl;
    for (int i = 1; i <= n; i++) {
        cout << "*" << " ";
    }
    cout << endl;

    cout << "Column:" << endl;
    for (int i = 1; i <= n; i++) {
        cout << "*" << endl;
    }
    return 0;
}
```

Output

```
Row:
* * * * *
Column:
*
*
*
*
*
```

2. Solid square

```
#include <iostream>
using namespace std;

int main() {
    int n = 4;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cout << "*" << " ";
        }
        cout << endl;
    }
    return 0;
}
```

Output

```
* * * *
* * * *
* * * *
* * * *
```

3. Right-angled triangle

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

Output

```
*
* *
* * *
* * * *
* * * * *
```

Homework Problems

1. Inverted right-angled triangle

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n - i + 1; j++) {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

Output

```
* * * * *
* * * *
* * *
* *
*
```

2. Solid rectangle of R rows and C columns

```
#include <iostream>
using namespace std;

int main() {
    int rows = 3, cols = 6;

    for (int i = 1; i <= rows; i++) {
        for (int j = 1; j <= cols; j++) {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

Output

```
* * * * * *
* * * * * *
* * * * * *
```

Lesson 3: Pyramids

Quick recap: Pyramids • Inverted pyramids • Managing spaces with stars

Theory & Nuances

Pyramids

A pyramid is a triangle centered on the screen. Row i (1-based, n rows) has $n - i$ leading spaces followed by $2i - 1$ stars, so the star count grows by 2 every row: 1, 3, 5, 7 ... Each row therefore needs two inner loops one after the other — first the spaces, then the stars.

Inverted pyramids

Run the same formulas with the rows reversed: row i has $i - 1$ leading spaces and $2(n - i) + 1$ stars. Equivalently, let the outer loop go from n down to 1 and reuse the exact pyramid body.

Managing spaces with stars

Leading spaces are what push stars to the right — the computer has no idea of "center", it just prints characters left to right. Build a table with three columns (row, spaces, stars) and find the formula for each column. A useful trick: printing "*" (2 characters wide) with $n - i$ single spaces in front also gives a centered pyramid, with only i stars per row.

Key Points / Common Mistakes

- Spaces loop first, stars loop second — and both live inside the same outer row loop.
- Trailing spaces after the stars are invisible, so you never need to print them.
- For a pyramid of n rows, the last row has $2n - 1$ stars, so the pattern is $2n - 1$ characters wide.

Classwork Problems

1. Star pyramid

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++) {
            cout << " ";
        }
        for (int j = 1; j <= 2 * i - 1; j++) {
            cout << "*";
        }
        cout << endl;
    }
    return 0;
}
```

Output

```
  *
 ***
*****
*****
*****
```

2. Inverted pyramid

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = n; i >= 1; i--) {
        for (int s = 1; s <= n - i; s++) {
            cout << " ";
        }
        for (int j = 1; j <= 2 * i - 1; j++) {
            cout << "*";
        }
        cout << endl;
    }
    return 0;
}
```

Output

```
*****
*****
****
***
**
*
```

3. Palindromic number pyramid

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++) {
            cout << " ";
        }
        for (int j = 1; j <= i; j++) {
            cout << j;
        }
        for (int j = i - 1; j >= 1; j--) {
            cout << j;
        }
        cout << endl;
    }
    return 0;
}
```

Output

```
    1
   121
  12321
 1234321
123454321
```

Homework Problems

1. Right-aligned triangle

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++) {
            cout << " ";
        }
        for (int j = 1; j <= i; j++) {
            cout << "*";
        }
        cout << endl;
    }
    return 0;
}
```

Output

```
  *
 **
***
****
*****
```

2. Pyramid using spaced stars

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    // Each "* " is 2 characters wide, so n - i single spaces
    // are enough to center row i, which holds just i stars.
    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++) {
            cout << " ";
        }
        for (int j = 1; j <= i; j++) {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

Output

```
  *
 * *
* * *
* * * *
* * * * *
```

Lesson 4: Diamonds, Hollow & Alphabet Patterns

Quick recap: *Full diamond* • *Hollow patterns* • *Alphabet patterns*

Theory & Nuances

Full diamond

A diamond is a pyramid of n rows stacked on top of an inverted pyramid of $n - 1$ rows. The middle (widest) row must be printed only once — if the bottom half also has n rows, the widest row appears twice and the diamond gets a flat middle.

Hollow patterns

Start from the solid version of the shape, then inside the inner loop decide per cell: print a star only if the cell is on the boundary, otherwise print a space. For a square the boundary is $i == 1 \parallel i == n \parallel j == 1 \parallel j == n$. For a pyramid it is the first star of the row, the last star of the row, or any star on the last row.

Alphabet patterns

Characters are stored as numbers, so 'A' + 1 is 'B'. Printing `char('A' + j - 1)` in place of `j` turns any number pattern into an alphabet pattern. The cast to char matters — without it, `cout` prints the numeric code (66) instead of the letter.

Key Points / Common Mistakes

- In a hollow pattern, the inner cells still have to print a space — skipping them shifts the right border left.
- `cout << 'A' + 1` prints 66, not B. Write `char('A' + 1)`.
- Build complex shapes from halves you already know (diamond = pyramid + inverted pyramid) instead of deriving one big formula.

Classwork Problems

1. Full diamond

```
#include <iostream>
using namespace std;

int main() {
    int n = 4;

    // Top half: pyramid of n rows
    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++) cout << " ";
        for (int j = 1; j <= 2 * i - 1; j++) cout << "*";
        cout << endl;
    }

    // Bottom half: inverted pyramid of n - 1 rows
    for (int i = n - 1; i >= 1; i--) {
        for (int s = 1; s <= n - i; s++) cout << " ";
        for (int j = 1; j <= 2 * i - 1; j++) cout << "*";
        cout << endl;
    }
    return 0;
}
```

Output

```
  *
 ***
*****
*****
  ***
  *
```

2. Hollow square

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (i == 1 || i == n || j == 1 || j == n) cout << "*" << " ";
            else cout << "  ";
        }
        cout << endl;
    }
    return 0;
}
```

Output

```
* * * * *
*       *
*       *
*       *
*       *
* * * * *
```

3. Alphabet triangle

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cout << char('A' + j - 1) << " ";
        }
        cout << endl;
    }
    return 0;
}
```

Output

```
A
A B
A B C
A B C D
A B C D E
```

Homework Problems

1. Hollow pyramid

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++) {
            cout << " ";
        }
        for (int j = 1; j <= 2 * i - 1; j++) {
            if (j == 1 || j == 2 * i - 1 || i == n) cout << "**";
            else cout << " ";
        }
        cout << endl;
    }
    return 0;
}
```

Output

```
  *
 * *
*   *
*   *
*****
```

2. Alphabet pyramid

```
#include <iostream>
using namespace std;

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++) {
            cout << " ";
        }
        for (int j = 1; j <= i; j++) {
            cout << char('A' + j - 1);
        }
        for (int j = i - 1; j >= 1; j--) {
            cout << char('A' + j - 1);
        }
        cout << endl;
    }
    return 0;
}
```

Output

```
  A
 ABA
ABCBA
ABCD CBA
ABCDEDCBA
```
